Lecture 1. Introduction

Data science workflow, R, RStudio, tidyverse, running code, getting help

Contents

1.1 What the course is about **2**

1.1.1 Organisation and scope 2

1.2 Software **3**

1.3 Running R code **3**

1.4 Getting help **3**

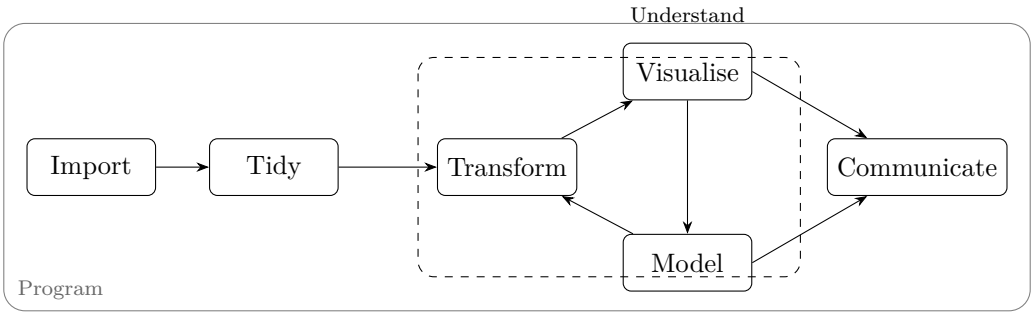
1.5 Selected exercises **4**

Based on the slides (*R for Data Science*, 1st ed., Wickham and Grolemund). All code was run at compile time; `##` lines are output. [Supplement] marks material not on the slides.

1.1 What the course is about

Motivation. Most of the work happens before the model: importing, cleaning and looking at data, done in reproducible code.

The course teaches how to collect, process, analyse and present data with a programming language (**data science**). A typical project:



- **Import**: file, database or web API → data frame.
- **Tidy**: *each column is a variable, each row an observation.*
- **Transform**: select observations, create variables, compute summaries. Tidy + transform = **wrangling**.
- **Visualise**: shows the unexpected and raises questions; needs a human, so does not scale.
- **Model**: answers a precise question; scales, but cannot check its own assumptions.
- **Communicate**: results matter only if others understand them.
- **Program**: runs through every stage; automates repeated tasks.

Visualise, transform and model form a loop traversed many times.

1.1.1 Organisation and scope

Order follows the book: visualisation and transformation first, then wrangling, programming and modelling.

Not covered: big data, languages other than R, non-rectangular data (images, sound, trees, text), and hypothesis *confirmation*. The course is about **hypothesis generation**.

Generation versus confirmation [Supplement]

An observation can suggest a hypothesis or test it, never both: testing a pattern on the data that suggested it is biased toward success (Lecture 13).

1.2 Software

Motivation. R (the language), RStudio (the editor) and the tidyverse (packages with one shared design).

R comes from CRAN (mirror `https://cloud.r-project.org`). **RStudio** is an IDE: run code in the console pane or write scripts in the editor pane.

A **package** bundles functions, data and documentation. The **tidyverse** is a family of packages with a common design. Install once, load every session:

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Core packages:

package	role
`ggplot2`	graphics (Lecture 2)
`dplyr`	data transformation (Lectures 3, 6)
`tidyr`	tidying (Lecture 5)
`readr`	reading rectangular files (Lecture 5)
`tibble`	a modern data frame (Lecture 5)
`stringr`, `forcats`	strings and factors (Lectures 7, 8)
`purrr`	functional programming (Lecture 12)

Conflicts: `dplyr::filter()` masks `stats::filter()`; `package::function()` names a function explicitly. Update with `tidyverse_update()`. Data packages used:

```
install.packages(c("nycflights13", "gapminder", "Lahman"))
```

1.3 Running R code

Motivation. How code and output are displayed.

At the prompt `>`, R evaluates a line and prints the result:

```
1 + 2
```

```
## [1] 3
```

Code is shown without the prompt; output lines start with `##` (a comment, so safe to copy). `[1]` is the index of the first element on the line: basic objects are vectors.

```
seq(10, 100, by = 5)
```

```
##  [1]  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90  95
## [19] 100
```

Functions carry parentheses (`sum()`), other objects do not (`flights`); `dplyr::mutate()` denotes `mutate()` from `dplyr`.

1.4 Getting help

`?mean` opens the help page of `mean()`; `??regression` searches all help pages. Outside R, search the exact error message (Stack Overflow, R-bloggers).

A **reproducible example** (reprex) gives the packages, a small data set (`dput(head(df))` prints it as R code) and the shortest code showing the problem.

```
dput(head(mtcars[, 1:3], 3))
```

```
## structure(list(mpg = c(21, 21, 22.8), cyl = c(6, 6, 4), disp = c(160,
## 160, 108)), row.names = c("Mazda RX4", "Mazda RX4 Wag", "Datsun 710"
## ), class = "data.frame")
```

1.5 Selected exercises

Motivation. Selection bias, floating point, masking and tidiness.

Exercise 1.1 (exploring and confirming on the same data)

An analyst explores a data set, finds k candidate patterns, and tests each at level α. Suppose no pattern is real and the k p-values are independent and uniform on $[0, 1]$.
(a) Show that the probability that at least one test is significant is $1 - (1 - \alpha)^k$; evaluate it for $\alpha = 0.05$, $k = 20$.
(b) The analyst reports only the most striking pattern and "confirms" it with its p-value on the same data. Show that this confirmation rejects with probability $1 - (1 - \alpha)^k$, not α.
(c) Show that testing the selected pattern on new, independent data rejects with probability exactly α.

Solution [Supplement].

(a) $P(\text{none significant}) = P(\text{all } p_j > \alpha) = (1 - \alpha)^k$ by independence, so $P(\text{at least one}) = 1 - (1 - \alpha)^k$; for $\alpha = 0.05$, $k = 20$: $1 - 0.95^{20} = 0.642$.

(b) The reported p-value is $p_{(1)} = \min_j p_j$, and $p_{(1)} \le \alpha$ iff some $p_j \le \alpha$: probability $1 - (1 - \alpha)^k$ (64%, not 5%).

(c) Let J be the selected index and q_j the p-values on new data, independent of J and uniform under the null:

$$P(q_J \le \alpha) = \sum_j P(J = j)\, P(q_j \le \alpha \mid J = j) = \sum_j P(J = j)\, \alpha = \alpha.$$

■

Exercise 1.2 (why $0.1 + 0.2 \neq 0.3$)

Explain the output below. Show that $1/10$ has no finite binary expansion, and characterise the decimals that are stored exactly.

```
0.1 + 0.2 == 0.3
```

```
## [1] FALSE
```

```
sprintf("%.20f", 0.1)
```

```
## [1] "0.10000000000000000555"
```

```
isTRUE(all.equal(0.1 + 0.2, 0.3))
```

```
## [1] TRUE
```

Solution [Supplement].

Step 1. Doubles are $m/2^k$ (integers m, k, m of at most 53 bits); exactly these numbers have finite binary expansions.

Step 2. $1/10 = m/2^k \Rightarrow 2^k = 10m \Rightarrow 5 \mid 2^k$, impossible. So 0.1 is stored as the nearest double, slightly above 0.1 (20 digits above); 0.2, 0.3 are rounded too.

Step 3. The errors do not cancel: computed $0.1 + 0.2$ and stored 0.3 are different doubles, so `==` gives `FALSE`. Exact decimals are the $m/2^k$: 0.5, 0.25, 0.375.

Step 4. Compare with a tolerance: `all.equal()` (relative tolerance $\approx 1.5 \times 10^{-8}$), wrapped in `isTRUE()` since it returns a description, not `FALSE`, on a mismatch. ■

Exercise 1.3 (masking)

After `library(tidyverse)` the message says that `dplyr::filter()` masks `stats::filter()`. Explain the two results below.

```
stats::filter(1:5, rep(1, 3) / 3)
```

```
## Time Series:
## Start = 1
## End = 5
## Frequency = 1
## [1] NA  2  3  4 NA
```

```
filter(1:5, rep(1, 3) / 3)
```

```
## Error in UseMethod("filter"): no applicable method for 'filter' applied to an object of
    class "c('integer', 'numeric')"
```

Solution [Supplement].

Step 1. `stats::filter()` with weights $(1/3, 1/3, 1/3)$ is a centred moving average: $2, 3, 4$, with `NA` at both ends.

Step 2. A bare name is looked up along `search()`, global environment → attached packages → base. `library()` attaches dplyr ahead of `stats`, so `filter` finds dplyr's first.

Step 3. dplyr's `filter()` has no method for numeric vectors, hence the error. Writing `package::function()` bypasses the search path. ■

Exercise 1.4 (what is tidy; adapted from R4DS, Exercise 12.2.1.1)

A table has one row per country and columns `country`, `1999` and `2000`, the last two holding the number of tuberculosis cases in that year. Name the variables and the observations, explain why the table is not tidy, and describe the tidy table for c countries.

Solution [Supplement].

Step 1. Variables: country, year, cases; observation: one country in one year. Values of year are column names and cases is spread over two columns, so a row holds two observations.

Step 2. Tidy: columns `country`, `year`, `cases`, one row per country–year, $2c$ rows; its $2c$ values of `cases` are the $2c$ year-column entries, so nothing is lost (`gather()`/`pivot_longer()`, Lecture 5). ■